

Phase 0: The Initiation

`scheduler.py` ➡ `main.py` ➡ `auth_check.py`

1. The Scheduler activates at a designated time slot (e.g., `4_PM`).
 2. It launches `main.py --auto`.
 3. The **Pre-Flight Check** immediately verifies your `token.pickle`. If it's expired, it silently refreshes it. If the token is invalid, it aborts the entire pipeline to save your CPU from wasting time.
-

Phase 1: Content Acquisition

`core/scrapper.py`

1. **Niche Selection:** The pipeline identifies the niche for the current time slot (e.g., `war_news`).
 2. **Parallel RSS Fetch:** It accesses multiple RSS feeds concurrently (Al Jazeera, Defense News, etc.).
 3. **Topic Selection:** It grabs the most relevant breaking news article.
 4. **Deep Reading:** It attempts to use Jina AI to read the full article. If Jina hits a CAPTCHA or Cloudflare block, your **Native Fallback** kicks in and extracts the raw text directly using BeautifulSoup.
-

Phase 2: AI Processing

`core/brain.py`

1. **LLM Prompting:** The raw article is sent to the Groq API (Llama 3 / Mixtral).
 2. **Script Generation:** The AI writes a highly engaging, short-form English script.
 3. **Segmentation:** The AI breaks the script down into exact visual scenes.
 4. **Metadata Creation:** The AI generates the YouTube Title, Description, Tags, and specific search keywords for each scene. Everything is saved to your local MongoDB.
-

Phase 3: Asset Generation (Parallel)

`core/voice.py` & `core/visuals.py`

1. **Voice Synthesis:** The **VoiceEngine** sends the text to Edge TTS and downloads high-quality **.mp3** narration files for each scene.
 2. **Visual Scouting:** The **VisualScout** fires up its async engine. It hunts for visuals across Pexels, Pixabay, Wikimedia, NASA, and the Internet Archive *simultaneously*.
 3. **Standardization:** Any non-MP4 files are quickly converted to **.mp4** using local FFmpeg.
-

Phase 4: The Synthesis (The Heavy Lifting)

core/assembler.py

1. **Audio Timestamps:** The local **Whisper AI** model loads into RAM, listens to the **.mp3** files, and generates exact millisecond timestamps for every single word. Whisper then unloads to free up RAM.
 2. **Semantic Syncing:** **RapidFuzz** reads the AI's "trigger words" and matches them to Whisper's timestamps, deciding exactly when to cut to the next video clip.
 3. **Visual Assembly:** The script crops all downloaded visuals to vertical (1080x1920), applies color grading and Ken Burns effects (zoom/pan), and stitches them together.
 4. **Final Polish:** Background music is added at 10% volume, and the bold yellow/white captions are burned directly onto the video.
 5. **Output:** **FINAL_VIDEO0.mp4** is saved to your hard drive.
-

Phase 5: Delivery & Cleanup

core/upload_prep.py & **core/uploader.py**

1. **Packaging:** **upload_prep.py** bundles the AI-generated Title, Description, and Tags into a **FINAL_VIDEO0_METADATA.txt** file next to the video.
2. **YouTube API:** **uploader.py** authenticates with Google and pushes the video file in 4MB chunks to YouTube as a private/scheduled Short.
3. **Cleanup:** Once YouTube returns a successful upload confirmation and a Video ID, the script deletes the temporary metadata text file.
4. **Logging:** A permanent, lightweight **0(1)** log is written to **production_log.jsonl** so you can track your channel's history.

 **Pipeline Complete.** The system goes back to sleep until the next scheduled slot!

[RUN: python main.py]

① PRE-FLIGHT GUARDRAIL —————> (Fails?) —> [System Exit / Clean Crash]

(auth_check.py)
| (Passes)

② MODE SELECTION & ARG PARSING

[--auto]

[--manual]

③ AUTO TOPIC SCRAPING

(core/scrapper.py)

- Fetches 4 RSS feeds
- Native Fallbacks

③ MANUAL TOPIC SELECTION

(User Input or RSS Choices)

- Shows 3 viral options or accepts custom topic
- "Is this good? (Y/N)"

④ AUTO SCRIPT GEN

(core/brain.py)

- AI writes & segments
- Saves to Database

④ MANUAL SCRIPT REVIEW

(core/brain.py)

- AI writes script
- User can type feedback to regenerate

(The paths merge into the final assembly line)

⑤ POST-SCRIPT PIPELINE (The Factory Floor)

- ➡ 🗣️ Voice Engine (core/voice.py)
 - Edge TTS generates parallel .mp3s
 - Updates DB: "voiced"
- ➡ 🎬 Visual Scout (core/visuals.py)
 - Async hunting on Pexels/Pixabay/NASA
 - Updates DB: "ready_to_assemble"
- ➡ 🎞️ Video Assembler (core/assembler.py)
 - RapidFuzz + Whisper syncs clips to words
 - Applies Ken Burns & Color Grading
 - Updates DB: "ready_to_upload"
- ➡ 📦 Upload Manager (core/upload_prep.py)

- Bundles AI Title, Desc, and Tags into txt file
- Updates DB: "completed_package"

▶ 🚀 YouTube Uploader (core/uploader.py)

- Grabs oldest "packaged" video from DB
- Uploads chunk-by-chunk to YouTube
- Updates DB: "uploaded"
- Deletes root metadata artifact

⑥ LOGGING & COMPLETION

- Writes 1-line JSON summary to `production\_log.jsonl`
- Graceful Exit

